Garbage Collection

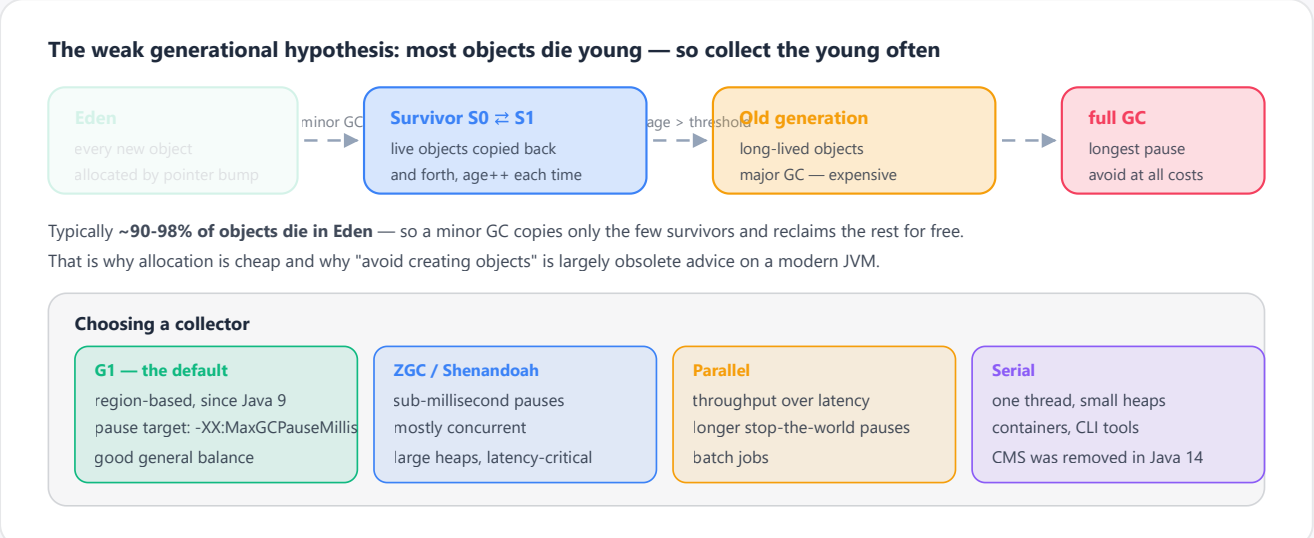
Why it matters

GC is the JVM's headline feature and the source of its most-discussed weakness: pauses. Understanding generational collection explains why allocation is cheap, why "avoid creating objects" is largely obsolete advice, and why one application benefits from G1 while another needs ZGC. It is also the vocabulary for every latency investigation you will ever run on a JVM.

The concept, simply

The collector reclaims objects that are no longer **reachable** from a *GC root* — stack references, static fields, JNI references, active threads. Reachability, not usefulness, is the only criterion, which is why the leaks from Chapter 1 are possible.

The design rests on the **weak generational hypothesis**: most objects die very young.



Typically 90-98% of objects never leave Eden. A minor GC therefore copies only the few survivors and reclaims everything else for free — which is what makes allocation cheap.

Mark and sweep, then compact. The collector marks everything reachable from the roots, sweeps what is not, and compacts survivors so allocation stays a pointer bump. Because the young generation is collected by *copying* the few survivors out, the cost of a minor GC is proportional to what *lives*, not to what died — which is why creating many short-lived objects is nearly free.

The collectors, and when each is right:

Collector	Optimises for	Use when
G1 (default since 9)	Balance, with a pause target	Almost everything — start here
ZGC / Shenandoah	Sub-millisecond pauses	Large heaps, latency-critical services
Parallel	Raw throughput	Batch jobs where pauses do not matter
Serial	Small footprint	Small containers, short-lived processes

G1 divides the heap into equal regions rather than contiguous generations, collects the regions with the most garbage first — hence "garbage first" — and works toward a pause-time target you set. **ZGC and Shenandoah** do nearly all their work concurrently with the application, keeping pauses under a millisecond largely independent of heap size. **CMS was removed in Java 14**, which is worth knowing because a lot of older tuning advice still recommends it.

Code examples

BASH

Copy

```
# Choosing a collector
java -XX:+UseG1GC -XX:MaxGCPauseMillis=200 -jar app.jar   # default; set a
java -XX:+UseZGC -jar app.jar                             # latency-criti
java -XX:+UseParallelGC -jar app.jar                      # batch through
java -XX:+UseSerialGC -jar app.jar                        # small contain
```

```
java -Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=10M

# Dump the heap automatically when it blows up. The single most valuable
# production flag in this area — without it you get one line in a log and
# no way to find the cause.
java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/log/heap.hprof
```

Reading a GC log, which is the actual skill:

TEXT Copy

```
[2.456s][info][gc] GC(12) Pause Young (Normal) (G1 Evacuation Pause)
                        512M→48M(1024M) 8.234ms

// Read it as: heap was 512M before, 48M after, out of a 1024M total, and it
// took 8ms. Healthy — nearly everything collected, short pause.

[45.2s][info][gc] GC(89) Pause Full (G1 Compaction Pause)
                        980M→940M(1024M) 2340.567ms

// Unhealthy: a FULL GC, 2.3 SECONDS of stop-the-world, and it only freed
// 40M of 980M. The heap is genuinely full of live objects — a leak or an
// undersized heap. Frequent full GCs that reclaim little are the classic
// signature of a memory leak.
```

Reference types, which is a common follow-up:

JAVA Copy

```
Object strong = new Object();                    // never collected while

SoftReference<byte[]> soft = new SoftReference<>(data);
// Collected only when memory is tight → plausible for caches, though in
// practice a bounded cache (Caffeine) is far more predictable.
```

```
// Collected at the next GC once no strong reference remains. This is what
// WeakHashMap uses for keys — the standard fix for listener-registry leak

PhantomReference<Resource> phantom = new PhantomReference<>(r, queue);
// For cleanup after collection. Replaces finalize(), which is deprecated
// for removal — finalizers were unpredictable, could resurrect objects, a
// delayed collection. Use Cleaner, or just try-with-resources.
```

Under the hood

Stop-the-world pauses are unavoidable in part. Even mostly-concurrent collectors must briefly pause all application threads to scan roots and finish marking. Pausing requires every thread to reach a *safepoint* — a known-good instruction boundary — so a thread in a long counted loop can delay the whole pause. That is why an occasional pause is far longer than the collector's target and shows as "time to safepoint" in the logs.

Tuning advice, in the order that actually works: first fix allocation and leaks in the application, because no collector rescues code that allocates faster than it can collect. Then size the heap correctly. Then choose the right collector for the workload. Only then touch individual flags — hand-tuning generation sizes is rarely productive with G1, which adapts. Most "GC problems" are application problems.

Never call `System.gc()`. It is a hint, it typically forces a full stop-the-world collection, and it usually makes things worse. Production JVMs frequently run with `-XX:+DisableExplicitGC` for exactly this reason.

`finalize()` **is deprecated for removal**, and it deserves the reputation: finalizers ran on an unpredictable schedule, could resurrect objects, delayed collection by an extra cycle, and silently swallowed exceptions. Use try-with-resources for deterministic cleanup, and `java.lang.ref.Cleaner` for the rare case where a safety net is genuinely needed.

Common mistakes

- **Calling** `System.gc()`. Forces a full pause and rarely helps.

- **Believing a GC prevents leaks.** It frees unreachable objects; a static map keeps things reachable forever.
- **Tuning flags before fixing the application.** Most GC problems are allocation or leak problems.
- **Avoiding object creation for its own sake.** Short-lived objects are nearly free; the cost is what survives.
- **Using `finalize()`.** Deprecated for removal — use try-with-resources or `Cleaner`.
- **`SoftReference` caches.** Behaviour depends on memory pressure and is hard to reason about; use a bounded cache.
- **Running without GC logs.** They are cheap and are the first thing you will want.
- **Copying tuning flags from a blog.** Much of that advice targets CMS, removed in Java 14.

Interview lens

> **Q: "How does garbage collection work?"**
> A: The collector finds objects reachable from GC roots — stack references, statics, JNI handles, live threads — marks them, and reclaims the rest. The heap is generational because most objects die young: new objects go into Eden, survivors are copied between survivor spaces and aged, and long-lived ones are promoted to the old generation. Minor collections are frequent and cheap because their cost is proportional to what survives, not to what died.

> **Q: "Which collector would you choose?"**
> A: G1 by default — it is the default since Java 9, region-based, and takes a pause-time target. For a latency-sensitive service with a large heap I would move to ZGC or Shenandoah, which do almost everything concurrently and hold pauses under a millisecond regardless of heap size. Parallel for batch work where throughput matters and pauses do not, and Serial for small containers. CMS is not an option — it was removed in Java 14.

A: Start from the GC log: frequent full collections that reclaim very little mean the heap is genuinely full of live objects. Then take a heap dump — ideally automatically, via `-XX:+HeapDumpOnOutOfMemoryError` — and open it in Eclipse MAT. The dominator tree shows what retains the most memory, and the path-to-GC-roots shows what is holding it. That usually names the offending collection or listener within minutes.

Q: "What is a stop-the-world pause?"

A: A period when all application threads are halted so the collector can work safely. Even mostly-concurrent collectors need brief ones for root scanning and mark termination. Threads must reach a safepoint to be paused, so one in a long counted loop can delay everyone — which is why an occasional pause is much longer than the target and shows up as time-to-safepoint in the logs.

Q: "Should you call `System.gc()`?"

A: No. It is only a hint, it typically triggers a full stop-the-world collection, and it interferes with the collector's own scheduling. If memory is a problem the answer is to find what is retained, not to ask for a collection. Many production JVMs run with `-XX:+DisableExplicitGC` so that a library calling it cannot cause a pause.

Q: "What are soft, weak and phantom references?"

A: Strong references keep objects alive. Soft references are cleared only when memory is tight, which sounds ideal for caches but is unpredictable in practice. Weak references are cleared at the next GC once no strong reference remains — that is what `WeakHashMap` uses for keys, and the standard fix for listener-registry leaks. Phantom references exist for post-collection cleanup and, with `Cleaner`, replace `finalize()`, which is deprecated for removal.

Cheat-card

- Reclaims what is **unreachable from GC roots** — reachable ≠ useful.
- **Weak generational hypothesis**: most objects die in Eden → minor GC copies only survivors.
- **G1** default (region-based, pause target) · **ZGC/Shenandoah** sub-ms · **Parallel** throughput · **Serial** small. **CMS removed in 14.**
- Healthy log line: big drop, short pause. **Frequent full GCs reclaiming little = leak.**
- Always run with **GC logs** + `-XX:+HeapDumpOnOutOfMemoryError`.
- Diagnose with **Eclipse MAT**: dominator tree + path to GC roots.
- **Never** `System.gc()`. Fix the application before touching flags.
- Safepoints: a thread in a long loop can extend a pause for everyone.
- References: strong · soft (memory-pressure) · weak (`WeakHashMap`) · phantom (`Cleaner`). `finalize()` **is dead.**

QUICK CHECK — no marks, just for you.

1. What is the weak generational hypothesis, and why does it shape GC design?

- ○ Most objects die very young, so collecting the young generation frequently is cheap — a minor GC copies only the few survivors and reclaims the rest for free
- ○ Objects should be allocated in generations to reduce fragmentation
- ○ Garbage collection should run on a fixed schedule
- ○ Older objects reference newer ones, so the old generation must be scanned first

Check answer

2. Which GC log pattern indicates a memory leak?

- ○ A sawtooth heap graph that returns to a stable baseline

Frequent FULL collections that reclaim very little — the heap is genuinely full of live objects

- Long pauses during application startup
- Frequent minor collections that reclaim most of the young generation

Check answer

3. Which statement about collectors is currently accurate?

- Serial GC is the only option in containers
- CMS is the recommended low-pause collector
- G1 is the default since Java 9; ZGC and Shenandoah target sub-millisecond pauses; CMS was REMOVED in Java 14
- Parallel GC is the default and best for latency

Check answer

‹ Previous
JVM Memory Areas

Next ›
Diagnosing Memory and Performance Problems

LEARN MORE

ARTICLES & DOCS

HotSpot garbage collection tuning guide
docs.oracle.com

Eclipse Memory Analyzer (MAT)
eclipse.dev

VIDEOS



Java's G1 Garbage Collector
Java



All 7 Java Garbage Collectors Explained | Serial, Parallel, G1, ZGC, Shenandoah & More
CyberJAR

Hand-picked references for this lesson.